SOFTWARE REQUIREMENT ANALYSIS

Course Title : Software Project Design and Development
Course Code : CSE 416

Submitted To : Mr. Al-Imtiaz, Associate Professor & Head, University of Information Technology & Sciences (UITS)

Submitted By:
Name : Muhammad Imtiaz Ahmed
ID : 0432410005101138
Section : 7B

Name : Foyze Ahmed
ID : 1944651070
Section : 7B

Introduction

CodeLearn is an online marketplace platform designed to **effectively connect aspiring programming students** with **qualified technology teachers.** The platform aims to streamline the educational process by providing features for easy **session bookings, real-time communication between users, and comprehensive management of the entire lifecycle of educational sessions.** This report outlines the necessary **functional and non-functional** requirements to ensure the successful development and deployment of the CodeLearn platform.

Functional Requirements

The functional requirements define the specific actions the system must perform to deliver value to its users (Students, Teachers, and Admin).

User Authentication & Authorization

- Students must be able to **register and login** to the platform.
- Teachers must be able to **register and login** to the platform.
- Admin must be able to **login** via a separate administrative panel.
- The system shall implement **JWT-based authentication** for secure session management.
- The system shall enforce **Role-based access control** (student/teacher/admin) to manage feature visibility and permissions.

Teacher Management

- Teachers shall be able to **submit course/teaching requests** detailing their qualifications and proposed subjects.
- The Admin shall be able to **view, approve, or reject teacher requests** from the admin dashboard.
- Only **approved teachers** shall appear in the public teacher listing for students to browse.
- Teachers shall have the capability to **manage and update their profiles**, including availability and rates.
- The platform must support teaching in **multiple programming languages** (e.g., Python, JavaScript, C++, PHP, Kotlin).

Session Booking

- Students must be able to **browse available teachers** via a dedicated listing page.
- Students can **view detailed teacher profiles** (qualifications, reviews, available courses).
- Students must be able to **book educational sessions** with their selected teachers.
- The system must support the booking of both **online and in-person sessions**.
- The platform shall integrate **multiple payment methods** (bKash, Card, Bank Transfer) for session payments.
- The system must provide **session status tracking** (pending/confirmed/completed/cancelled).
- Students shall be able to **complete orders** once the session is delivered.
- Teachers shall be able to **cancel orders** under specific platform-defined conditions.

Real-time Chat System

- Students and Teachers shall be able to **chat with each other** in real-time.
- Students shall be able to **chat with admin support** for platform assistance.
- The chat system must ensure **real-time message updates** (via polling).
- The system must track and display **read receipts and unread message counts**.
- All chat conversations must feature **chat history preservation**.

Dashboard Functionality

- A **Student dashboard** providing an overview of their **order history** and upcoming sessions.
- A **Teacher dashboard** including:
    - Course/teaching requests status.
    - Direct access to Student chats.

    - Current and past Orders/sessions.
    - **Earnings calculation and history**.
- An **Admin dashboard** providing oversight for:
    - Teacher requests management (approval/rejection).
    - Order and session management.
    - Management of newsletter subscribers.
    - Session and platform analytics.

Additional Features

- A feature to allow users to **subscribe to the platform's newsletter**.
- Integration of a **Python code interpreter (Pyodide)** for interactive learning.
- Advanced **search and filter** capabilities for teachers based on programming language/category.
- An option for **random teacher selection** for students who are undecided.
- A mechanism to identify and **highlight the platform's best teachers** based on performance/reviews.

Non-Functional Requirements

The non-functional requirements specify criteria that can be used to judge the operation of the system, rather than specific behaviors.

Performance

- **Page Load Time:** All primary pages must load in **less than 3 seconds**.
- **API Response Time:** All critical API endpoints must respond in **less than 500ms**.
- **Concurrency:** The platform must reliably support a minimum of **1000+ concurrent users**.
- **Chat Polling:** The real-time chat must have a polling interval of **2 seconds**.

Security

- **Password Storage:** All user passwords must be secured using the **bcrypt hashing** algorithm.
- **Authentication:** Authentication must be strictly enforced via **JWT token-based authentication**.
- **API Protection:** The backend must implement **CORS protection** to restrict unauthorized access.
- **Data Integrity:** All user input must undergo rigorous **validation and sanitization**.
- **File Handling: Secure file upload** will be implemented using the Cloudinary service.

Usability

- **Design:** The application must utilize a **responsive design** to function seamlessly across mobile, tablet, and desktop devices.
- **Navigation:** The user interface must feature **intuitive navigation** and a clear information hierarchy.
- **Feedback:** The system shall provide **clear, constructive error messages** and visible **loading states and feedback** for user actions.

Scalability

- **Database:** Utilize **MongoDB** to leverage its flexible schema for evolving data structures.
- **Asset Management:** Use **Cloudinary** for scalable, off-server image and file storage.
- **Architecture:** The system must adopt a **modular architecture** to facilitate independent development and deployment of components.
- **API:** Implement a **RESTful API design** following standard best practices for easy client integration and future expansion.

Reliability

- **Fault Tolerance:** Comprehensive **error handling and logging** mechanisms must be implemented across the application.
- **Data Consistency:** Implement robust **transaction management** to ensure data consistency, particularly during booking and payment processes.
- **Disaster Recovery:** Establish periodic **data backup mechanisms** for all critical data.
- **User Experience:** Implement **graceful degradation** to ensure core functionality remains available even when non-critical components fail.